

Module 1: Lesson 3

Discrete Fourier Transform



Outline

- ▶ Discrete Fourier Transform (DFT)
- ▶ Fast-Fourier Transform (FFT)
- ▶ The FFT Algorithm in Practice

Discrete Fourier Transform (DFT)

We will now cover more in depth the **Discrete Fourier Transform (DFT)**, following the intuition developed in Carr and Madan (1999).

You may remember Carr and Madan's (1999) expression for the price of a Call option from Lesson 1:

$$C_0 = \frac{e^{-\alpha k}}{\pi} \int_0^{\infty} e^{-i\nu k} \Psi(\nu) d\nu$$

To solve this integral, however, we need to use some kind of **discretization technique**. For example, the **trapezoidal rule**:

$$\int_a^b f(x) dx \approx T(f, \eta) = \frac{\eta}{2} (f(x_0) + 2f(x_1) + 2f(x_2) + \dots + 2f(x_{N-2}) + 2f(x_{N-1}) + f(x_N))$$

Truncating the upper limit of the integral above to B , and applying the trapezoidal rule, yields the DFT:

$$C_T(k) = \frac{e^{-\alpha k}}{\pi} \int_0^{\infty} e^{-i\nu k} \Psi(\nu) d\nu \approx \frac{e^{-\alpha k}}{\pi} \int_0^B e^{-i\nu k} \Psi(\nu) d\nu \approx \frac{e^{-\alpha k}}{\pi} \sum_{j=1}^N e^{-i\nu_j k} \Psi(\nu_j) \eta \quad (1)$$

where $\nu_j = \eta(j-1)$ and $\eta = B/N$.

Fast Fourier Transform (FFT)

While the latter expression for DFT is relatively easy to compute, there is a faster way that decreases computational costs: the **Fast Fourier Transform (FFT)**.

The FFT maps a vector of the type $x = (x_j)_{j=1}^N$ to a vector $g(x_u)$ such that:

$$g(x_u) = \sum_{j=1}^N e^{-i \frac{2\pi}{N} (j-1)(u-1)} x_j, \text{ for } u = 1, \dots, N.$$

Then, we can create a range $[-b, b]$ for the *LogK* partitioned into N with λ equal spacing such that:

$$k_u = -b + \lambda(u-1), \text{ for } u = 1, \dots, N.$$

Substituting k_u and $\nu_j = \eta(j-1)$ in Eq. (1) yields

$$C_T(k_u) \approx \frac{e^{-\alpha k_u}}{\pi} \sum_{j=1}^N e^{-i \nu_j k_u} \Psi(\nu_j) \eta = \frac{e^{-\alpha k_u}}{\pi} \sum_{j=1}^N e^{-i \lambda \eta (j-1)(u-1)} e^{i b \nu_j} \Psi(\nu_j) \eta$$

where usually $\lambda \eta = 2\pi/N$ and $e^{i b \nu_j} \Psi(\nu_j) \eta = x_j$ in FFT. $C_T(k_u)$ gives the European call price for N strikes

FFT with Simpson's Rule

The trapezoidal rule is a valid and useful discretization technique. However, sometimes, the **Simpson rule** is preferred:

$$\int_a^b f(x) dx \approx S(f, \eta) = \frac{\eta}{3} (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + 2f(x_{2N-2}) + 4f(x_{2N-1}) + f(x_{2N}))$$

Applying Simpson's rule in the same spirit as before on Eq. (1) yields

$$C_T(k_u) = \frac{e^{-\alpha k_u}}{\pi} \sum_{j=1}^N e^{\frac{-i2\pi}{N}(j-1)(u-1)} e^{ib\nu_j} \Psi(\nu_j) \frac{\eta}{3} (3 + (-1)^j - \delta_{j-1})$$

where $\nu_j = \eta(j-1)$, $\eta = B/N$ and $\delta_n = \mathbb{I}_{n=0}$.

- ▶ This is the case for ITM options. For the OTM case, there are some stability issues that we will tackle in the future.
- ▶ This equation is implemented in the Python code from Lesson 1.

Wrap-up of Lesson 3

In Lesson 3, we have looked at:

- ▶ Discrete and Fast Fourier Transforms
- ▶ Discretization techniques: Trapezoidal and Simpson rules

⇒ **TO-DO NEXT:** Now you can go back to the Jupyter Notebook from Lesson 1, implementing FFT. You should be able to better understand the code now.

Please note that we focus mainly on the ITM/ATM case. The OTM case (for which you also have the code) requires a few extra tweaks to ensure integrability and numerical stability.

⇒ In the next lesson, we will look at how the semi-closed form solution in Lewis (2001) and FFT come in handy to perform a **calibration of Heston model** parameters.